EXPERIMENT – 4

Title

Joins, Subqueries, EXISTS, Simulated INTERSECT and EXCEPT, and EXPLAIN

Aim

To perform different SQL operations on the Employee–Department–Project schema using INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, simulated INTERSECT and EXCEPT, and to compare query execution plans using EXPLAIN.

Objectives

1. To understand and implement INNER JOIN.
2. To implement LEFT JOIN.
3. To perform a self-join on the Employee table.
4. To perform a 3-way join between Employee, Department, and Project.
5. To understand and implement correlated subqueries.
6. To use EXISTS for checking the existence of records.
7. To simulate INTERSECT using SQL conditions.
8. To simulate EXCEPT using NOT EXISTS.
9. To analyze query execution plans using EXPLAIN.

Software / Tools Required

- ByteXL / Nimbus
- MySQL
- SQL

Theory

A **JOIN** is used to combine data from two or more tables based on a related column. Joins are very useful when related information is stored in different tables.

In this experiment, the Employee, Department, and Project tables are used to retrieve related information.

Naman Verma
25LBCS3276

1. INNER JOIN

An **INNER JOIN** returns only those records that have matching values in both tables.

For example, Employee and Department can be joined using `DeptID`.

2. LEFT JOIN

A **LEFT JOIN** returns all records from the left table and the matching records from the right table. If there is no matching record, NULL values are returned for the right table.

3. Self-Join

A **self-join** is a join in which a table is joined with itself. It is useful for comparing records within the same table.

For example, employees having a higher salary than another employee can be compared using a self-join.

4. Three-Way Join

A **three-way join** combines three tables in a single query. In this experiment, Employee, Department, and Project are joined to display complete employee information.

5. Correlated Subquery

A correlated subquery is a subquery that depends on the value of the outer query. The subquery is evaluated for each row processed by the outer query.

6. EXISTS

The `EXISTS` operator checks whether a subquery returns at least one record. It returns TRUE when the subquery produces a result.

7. Simulated INTERSECT

INTERSECT returns records common to two result sets. When direct INTERSECT is not used, the same result can be simulated using `EXISTS` or a suitable JOIN.

Naman Verma
25LBCS3276

8. Simulated EXCEPT

EXCEPT returns records present in the first result set but not present in the second result set. It can be simulated using `NOT EXISTS`.

9. EXPLAIN

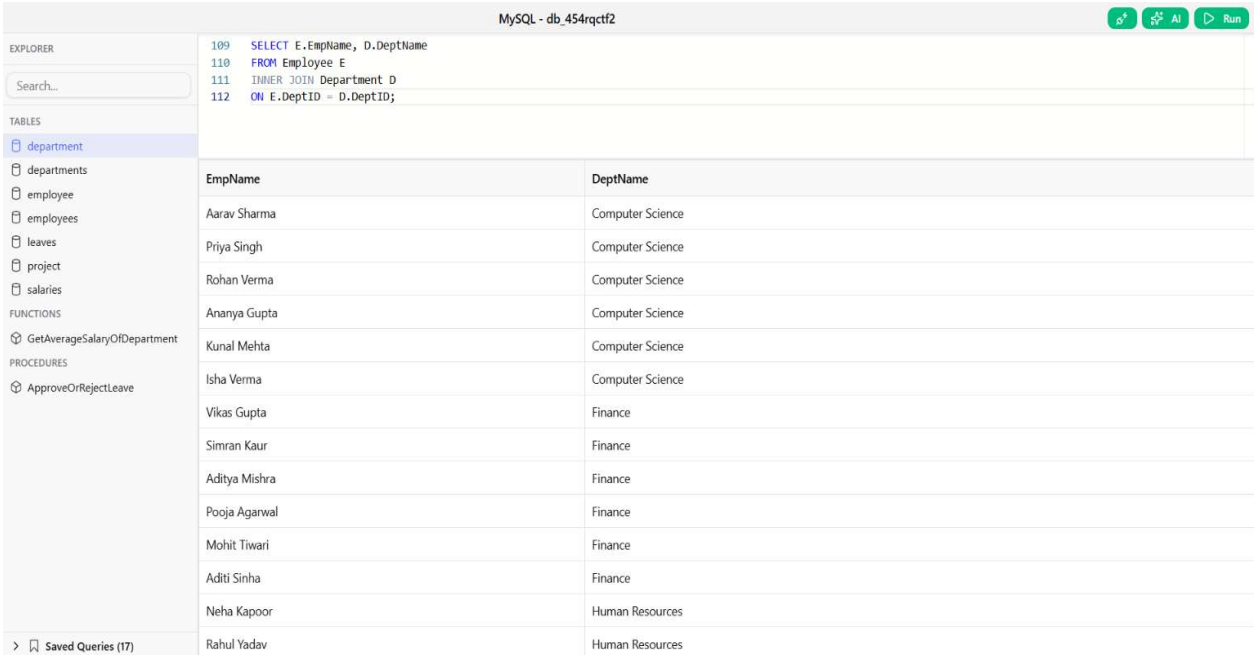
`EXPLAIN` is used to display the execution plan of an SQL query. It provides information about how MySQL plans to execute the query, including tables used, indexes, access type, and estimated rows.

SQL Implementation

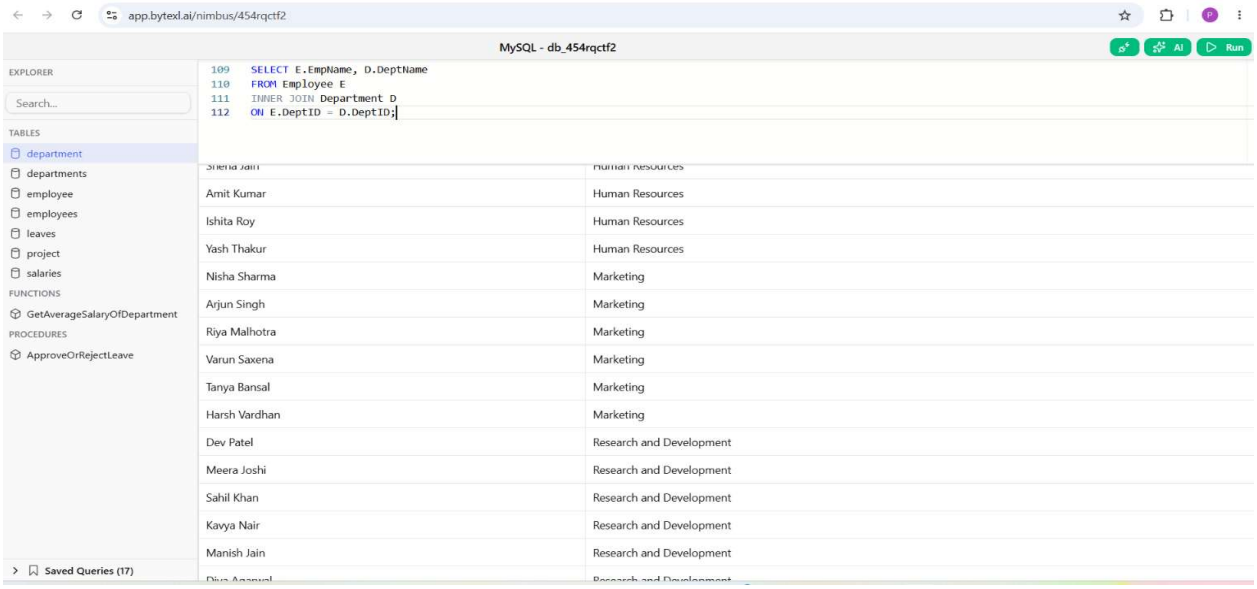
1. INNER JOIN

Question: Display employee names along with their department names.

```
SELECT E.EmpName, D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;
```



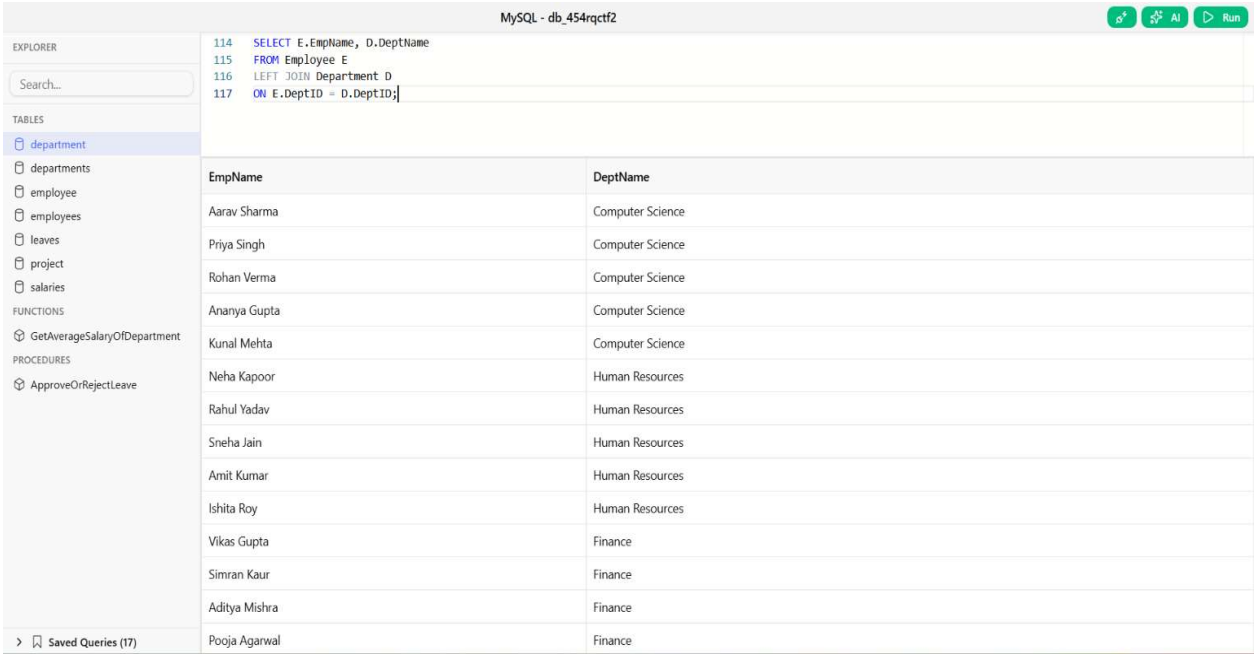
Naman Verma
25LBCS3276



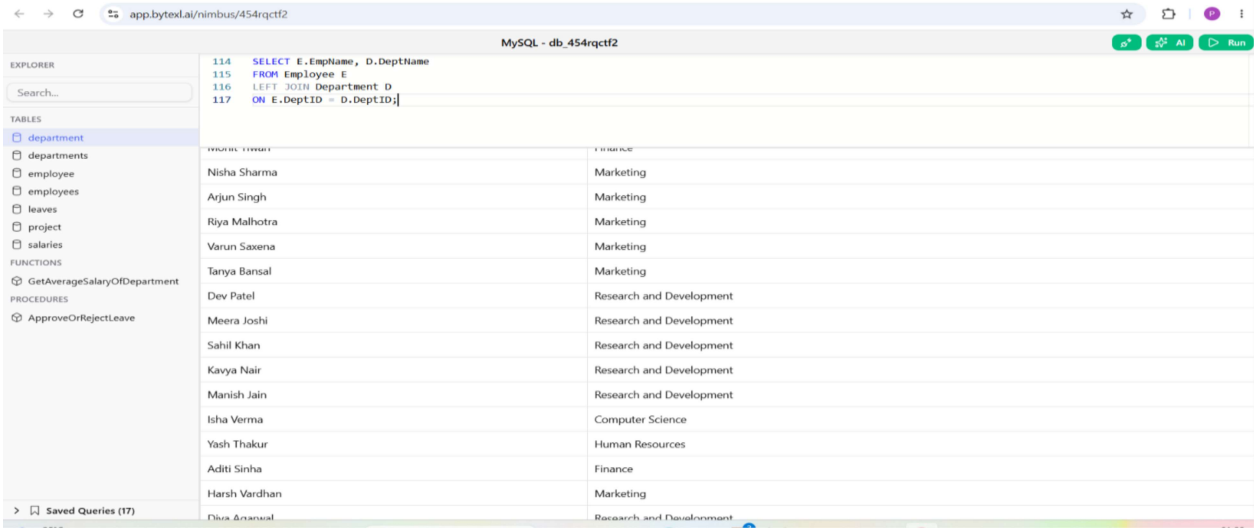
2. LEFT JOIN

Question: Display all employees and their department names.

```
SELECT E.EmpName, D.DeptName
FROM Employee E
LEFT JOIN Department D
ON E.DeptID = D.DeptID;
```



Naman Verma
25LBCS3276

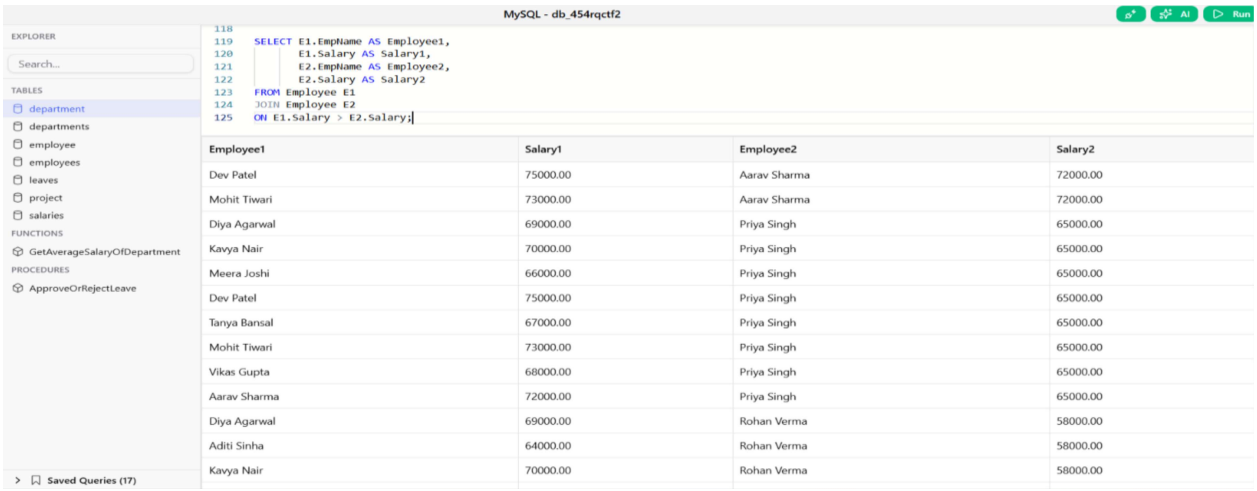


3. Self-Join

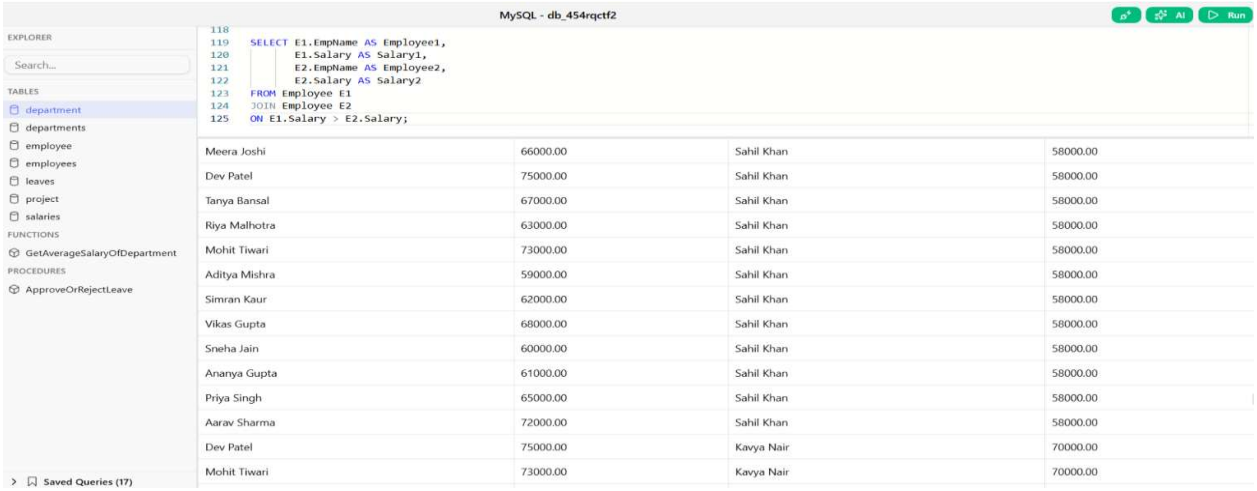
Question: Find pairs of employees where one employee has a higher salary than another employee.

```
SELECT E1.EmpName AS Employee1,
       E1.Salary AS Salary1,
       E2.EmpName AS Employee2,
       E2.Salary AS Salary2
FROM Employee E1 JOIN Employee E2
ON E1.Salary > E2.Salary;
```

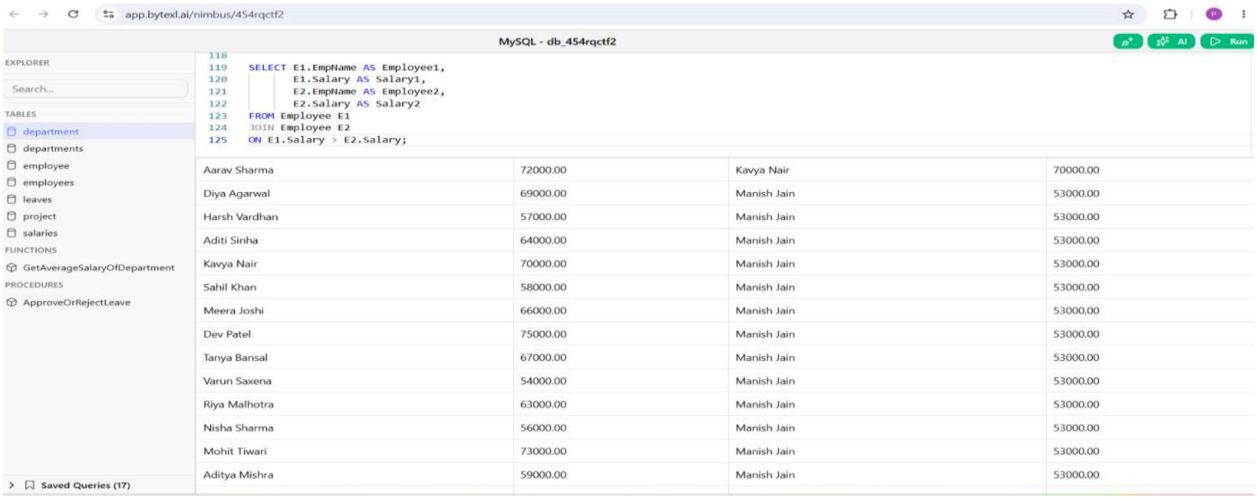
Note: This query can produce many rows because every possible salary comparison is considered.



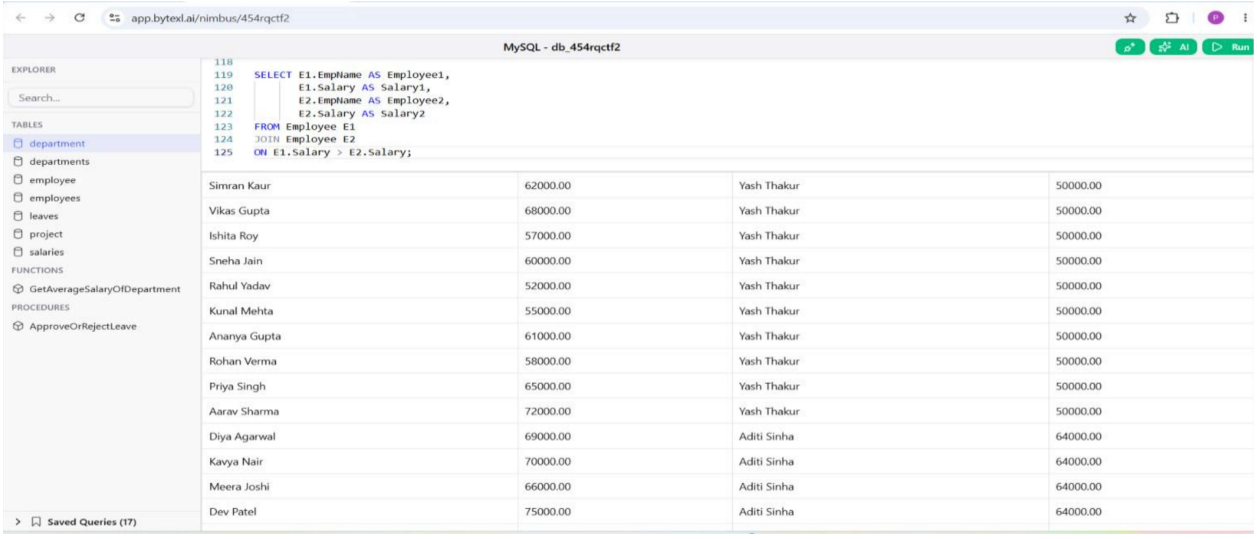
Naman Verma
25LBCS3276



Meera Joshi	66000.00	Sahil Khan	58000.00
Dev Patel	75000.00	Sahil Khan	58000.00
Tanya Bansal	67000.00	Sahil Khan	58000.00
Riya Malhotra	63000.00	Sahil Khan	58000.00
Mohit Tiwari	73000.00	Sahil Khan	58000.00
Aditya Mishra	59000.00	Sahil Khan	58000.00
Simran Kaur	62000.00	Sahil Khan	58000.00
Vikas Gupta	68000.00	Sahil Khan	58000.00
Sneha Jain	60000.00	Sahil Khan	58000.00
Ananya Gupta	61000.00	Sahil Khan	58000.00
Priya Singh	65000.00	Sahil Khan	58000.00
Aarav Sharma	72000.00	Sahil Khan	58000.00
Dev Patel	75000.00	Kavya Nair	70000.00
Mohit Tiwari	73000.00	Kavya Nair	70000.00

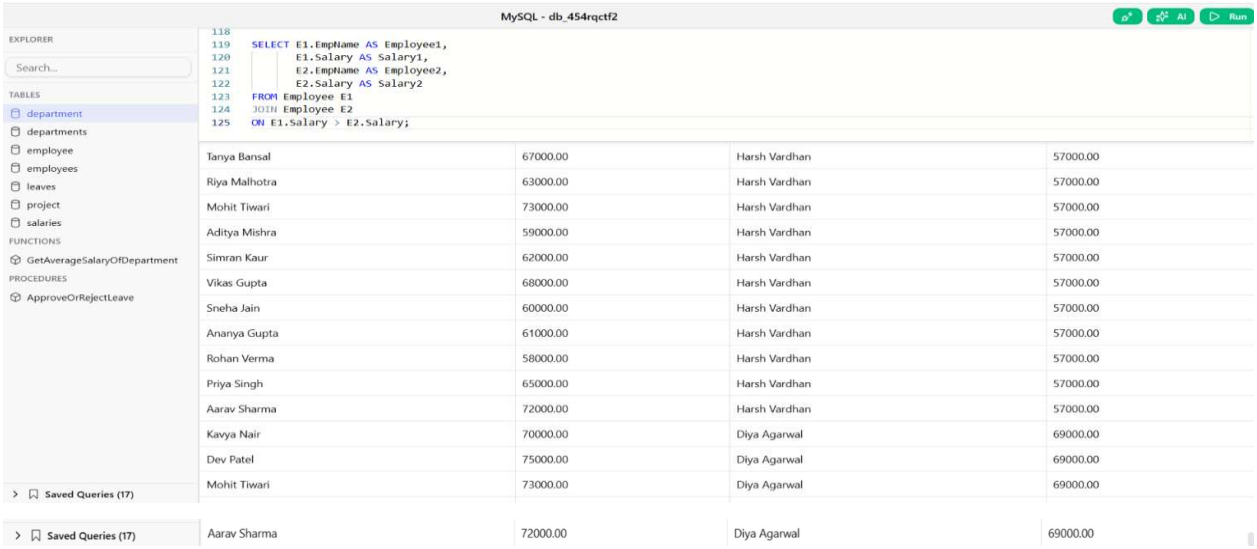


Aarav Sharma	72000.00	Kavya Nair	70000.00
Diya Agarwal	69000.00	Manish Jain	53000.00
Harsh Vardhan	57000.00	Manish Jain	53000.00
Aditi Sinha	64000.00	Manish Jain	53000.00
Kavya Nair	70000.00	Manish Jain	53000.00
Sahil Khan	58000.00	Manish Jain	53000.00
Meera Joshi	66000.00	Manish Jain	53000.00
Dev Patel	75000.00	Manish Jain	53000.00
Tanya Bansal	67000.00	Manish Jain	53000.00
Varun Saxena	54000.00	Manish Jain	53000.00
Riya Malhotra	63000.00	Manish Jain	53000.00
Nisha Sharma	56000.00	Manish Jain	53000.00
Mohit Tiwari	73000.00	Manish Jain	53000.00
Aditya Mishra	59000.00	Manish Jain	53000.00



Simran Kaur	62000.00	Yash Thakur	50000.00
Vikas Gupta	68000.00	Yash Thakur	50000.00
Ishita Roy	57000.00	Yash Thakur	50000.00
Sneha Jain	60000.00	Yash Thakur	50000.00
Rahul Yadav	52000.00	Yash Thakur	50000.00
Kunal Mehta	55000.00	Yash Thakur	50000.00
Ananya Gupta	61000.00	Yash Thakur	50000.00
Rohan Verma	58000.00	Yash Thakur	50000.00
Priya Singh	65000.00	Yash Thakur	50000.00
Aarav Sharma	72000.00	Yash Thakur	50000.00
Diya Agarwal	69000.00	Aditi Sinha	64000.00
Kavya Nair	70000.00	Aditi Sinha	64000.00
Meera Joshi	66000.00	Aditi Sinha	64000.00
Dev Patel	75000.00	Aditi Sinha	64000.00

Naman Verma
25LBCS3276



4. Three-Way Join

Question: Display employee name, department name, project name, and salary.

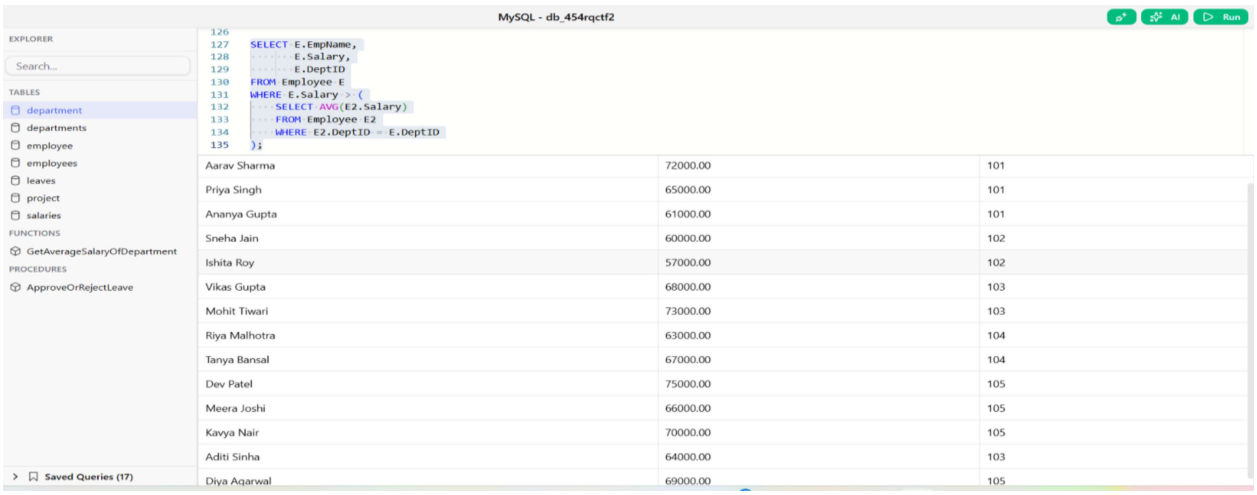
```
SELECT E.EmpName,
       D.DeptName,
       P.ProjectName,
       E.Salary
FROM Employee E
INNER JOIN Department D ON E.DeptID = D.DeptID
INNER JOIN Project P ON E.ProjectID = P.ProjectID;
```

5. Correlated Subquery

Question: Display employees whose salary is greater than the average salary of their own department.

```
SELECT E.EmpName,
       E.Salary,
       E.DeptID
FROM Employee E
WHERE E.Salary > (
    SELECT AVG(E2.Salary)
    FROM Employee E2
    WHERE E2.DeptID = E.DeptID
);
```

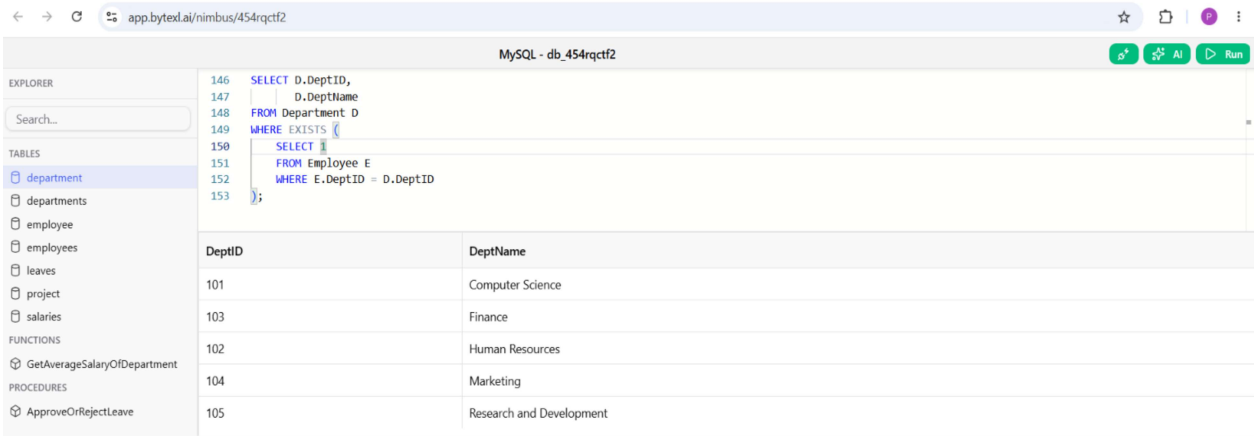
Naman Verma
25LBCS3276



6. EXISTS

Question: Display departments that have at least one employee.

```
SELECT D.DeptID,
       D.DeptName
FROM Department D
WHERE EXISTS (
    SELECT 1
    FROM Employee E
    WHERE E.DeptID = D.DeptID
);
```



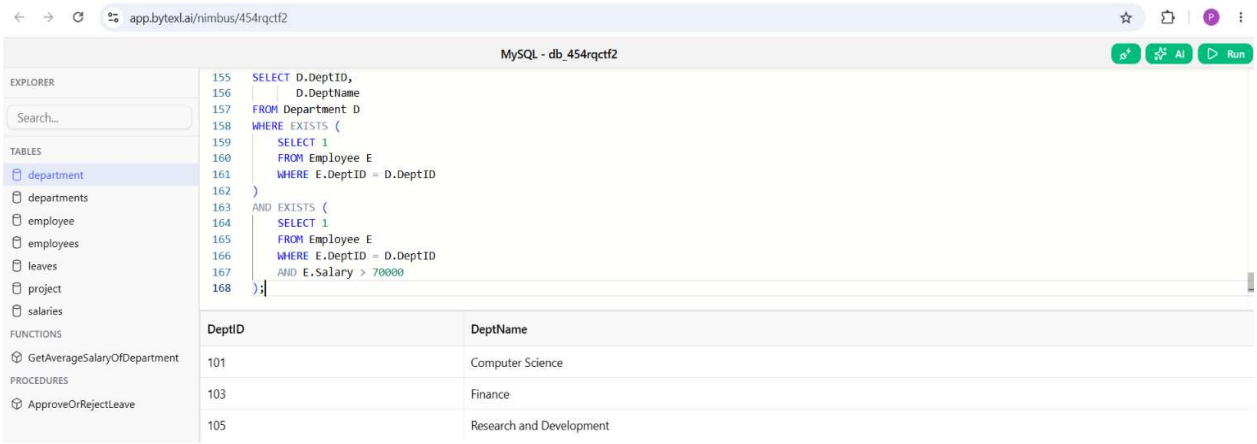
7. Simulated INTERSECT

Question: Find departments that have employees and are also associated with employees earning more than 70000.

Naman Verma
25LBCS3276

```
SELECT D.DeptID,
       D.DeptName
FROM Department D
WHERE EXISTS (
    SELECT 1
    FROM Employee E
    WHERE E.DeptID = D.DeptID
)
AND EXISTS (
    SELECT 1
    FROM Employee E
    WHERE E.DeptID = D.DeptID
    AND E.Salary > 70000
);
```

This returns departments common to both conditions and therefore simulates the intersection of the two result sets.



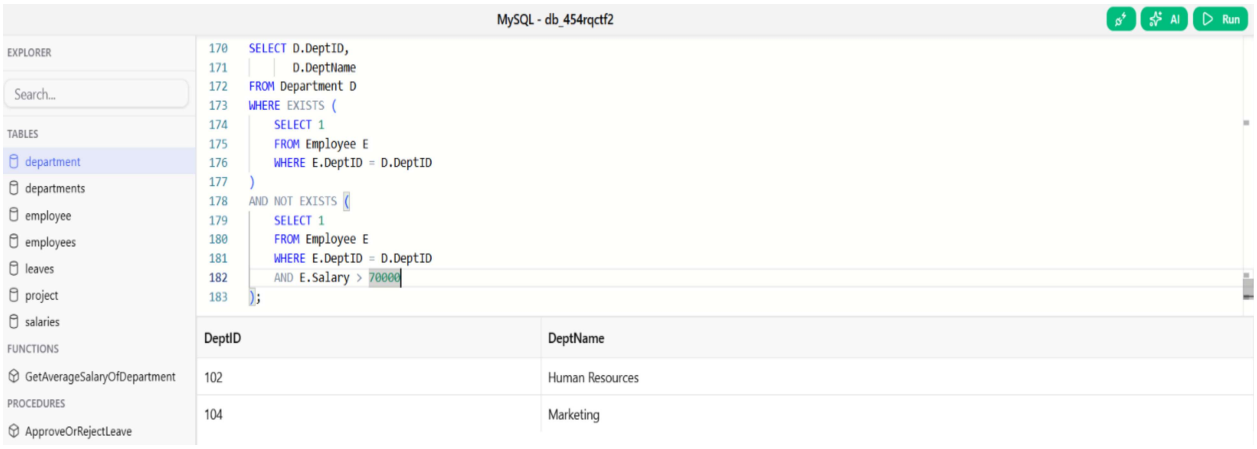
8. Simulated EXCEPT

Question: Find departments that have employees but do not have any employee earning more than 70000.

```
SELECT D.DeptID,
       D.DeptName
FROM Department D
WHERE EXISTS (
    SELECT 1
```

```
    FROM Employee E
    WHERE E.DeptID = D.DeptID
)
AND NOT EXISTS (
    SELECT 1
    FROM Employee E
    WHERE E.DeptID = D.DeptID
    AND E.Salary > 70000
);
```

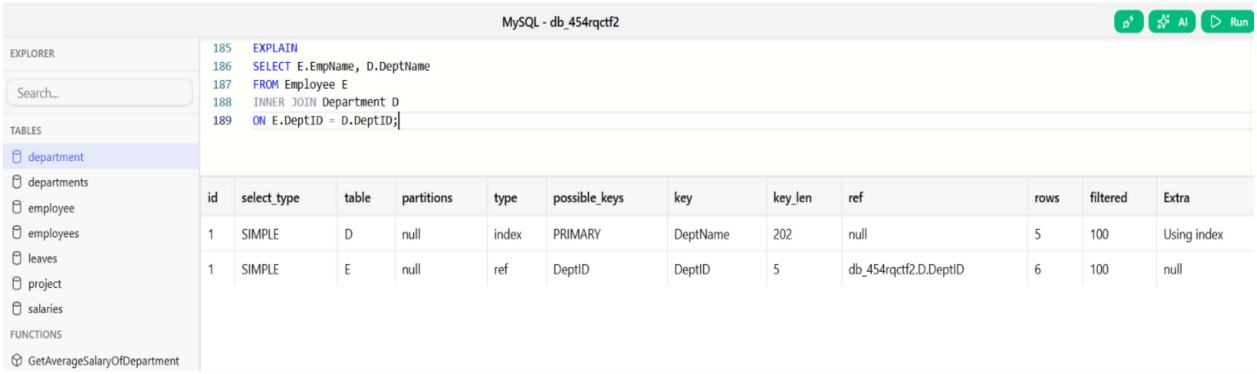
This returns departments satisfying the first condition but not the second condition and therefore simulates EXCEPT.



9. EXPLAIN – INNER JOIN

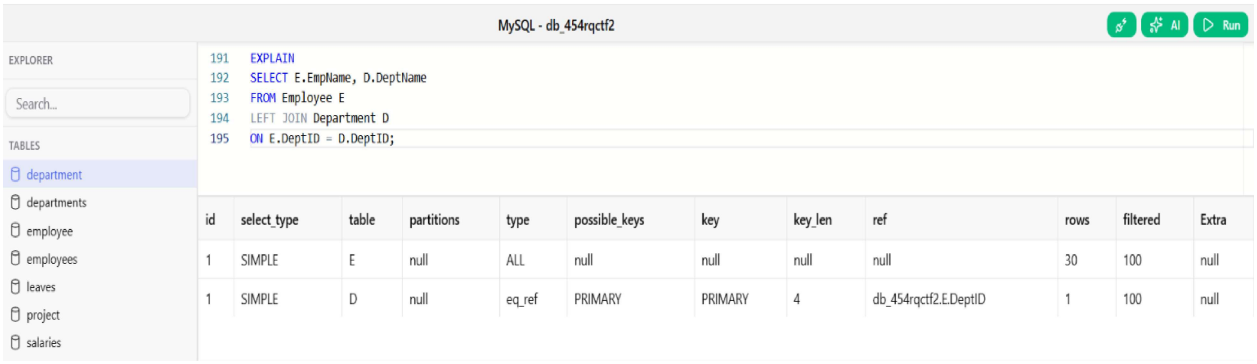
To examine the execution plan of the INNER JOIN:

```
EXPLAIN
SELECT E.EmpName, D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;
```



10. EXPLAIN – LEFT JOIN

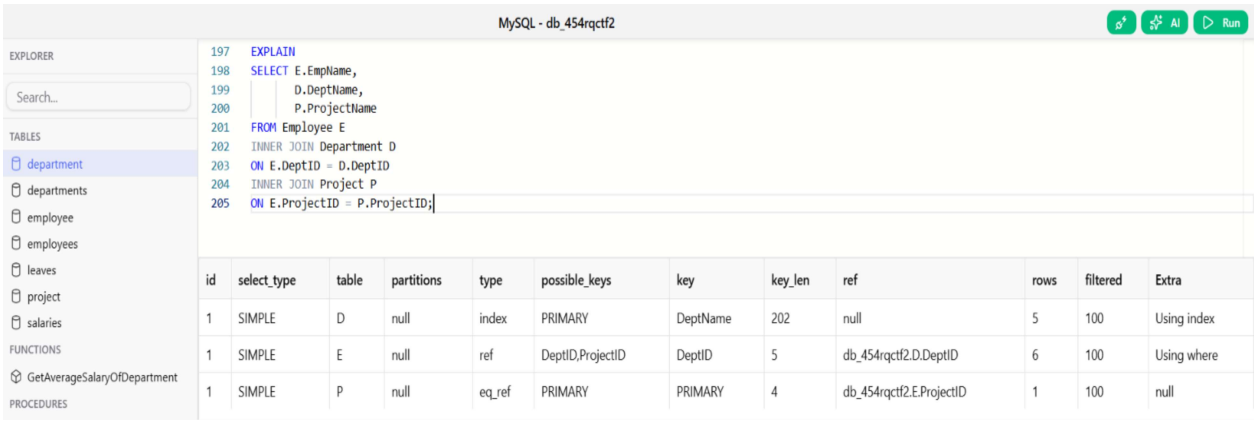
```
EXPLAIN
SELECT E.EmpName, D.DeptName
FROM Employee E
LEFT JOIN Department D
ON E.DeptID = D.DeptID;
```



11. EXPLAIN – Three-Way JOIN

```
EXPLAIN
SELECT E.EmpName,
       D.DeptName,
       P.ProjectName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID
INNER JOIN Project P
ON E.ProjectID = P.ProjectID;
```

Naman Verma
25LBCS3276



12. Comparison of Execution Plans

The `EXPLAIN` command can be used to compare the execution plans of different SQL queries.

The execution plan helps in understanding how the database processes a query. It can also help identify whether indexes are being used and how many rows are estimated to be examined.

Query	Purpose	Execution Plan Information
INNER JOIN	Combines matching employee and department records	Shows tables accessed and join method
LEFT JOIN	Returns all employees with matching departments	Shows left table and matching access
Three-Way JOIN	Combines three related tables	Shows access and join order for three tables

14. Result

The Employee–Department–Project schema was successfully used to implement INNER JOIN, LEFT JOIN, self-join, three-way join, correlated subqueries, EXISTS, simulated

Naman Verma
25LBCS3276

INTERSECT, and simulated EXCEPT. The EXPLAIN command was also used to examine and compare the execution plans of SQL queries.

15. Learning Outcomes

After completing this experiment, we are able to:

1. Use different types of SQL joins.
2. Compare records within the same table using self-join.
3. Combine three related tables using a three-way join.
4. Write and understand correlated subqueries.
5. Use EXISTS and NOT EXISTS.
6. Simulate INTERSECT and EXCEPT operations.
7. Analyze SQL execution plans using EXPLAIN.

16. Conclusion

This experiment provided practical knowledge of advanced SQL queries. Different types of joins, subqueries, EXISTS, simulated set operations, and EXPLAIN were successfully implemented on the Employee–Department–Project database schema.

Naman Verma
25LBCS3276